



武汉大学
WUHAN UNIVERSITY

《分布式软件原理与技术》

第八讲 服务治理 (2)

计算机学院 贾向阳



1、可观测性



什么是可观测性？

➤ 定义

- 可观测性是指通过观察系统的外部输出（日志、指标、链路追踪）来推断其内部状态的能力。

➤ 为什么微服务需要可观测性？

- 服务数量多（几十上百个），调用链复杂。
- 故障定位困难：一个请求失败，可能是网关、订单、库存、数据库任一环节出问题。
- 传统监控（检查CPU、内存）无法回答“为什么这个请求慢？”。

➤ 可观测性 ≠ 监控

- 监控：已知可能出现问题的地方，预先设置告警阈值。
- 可观测性：探索未知问题，通过数据主动发现异常。



三大支柱

支柱	作用	典型问题
日志 (Logging)	记录离散事件	"谁在什么时候做了什么?"
指标 (Metrics)	聚合统计数据	"每秒有多少请求? 错误率多少?"
链路追踪 (Tracing)	追踪请求路径	"请求经过了哪些服务? 每个服务耗时多少?"

可观测性允许你探索未知问题——比如 ‘为什么今天下午3点突然P99延迟过高?’ , 可以通过关联日志、指标和追踪来定位。”

请求 → Traceld=abc123

(2.分析延迟高的请求链路)

└─ Span A (订单服务, 1200ms)

└─ 日志: "收到下单请求, traceld=abc123"

└─ Span B (调用库存服务, 1100ms)

└─ 日志: "扣减库存, skuld=1001"

(3.分析日志定位问题)

└─ 指标: 订单服务 TPS=300, 延迟 P99=1200ms (1. 发现指标异常)



2、日志监控



日志

➤ 日志

- 最原始的操作事件记录
- 一定要带上 `traceld`，这样才能把同一次请求的日志串起来

➤ 格式

- 结构化日志 (JSON) 优于非结构化文本，便于检索
- 典型实现：ELK (Elasticsearch + Logstash + Kibana) 、Loki + Promtail。

➤ 关键实践

- 统一字段：timestamp、level、service、traceld、message。
- 避免日志泛滥：采样、异步输出、按级别过滤。



日志采集方案



日志采集的目标

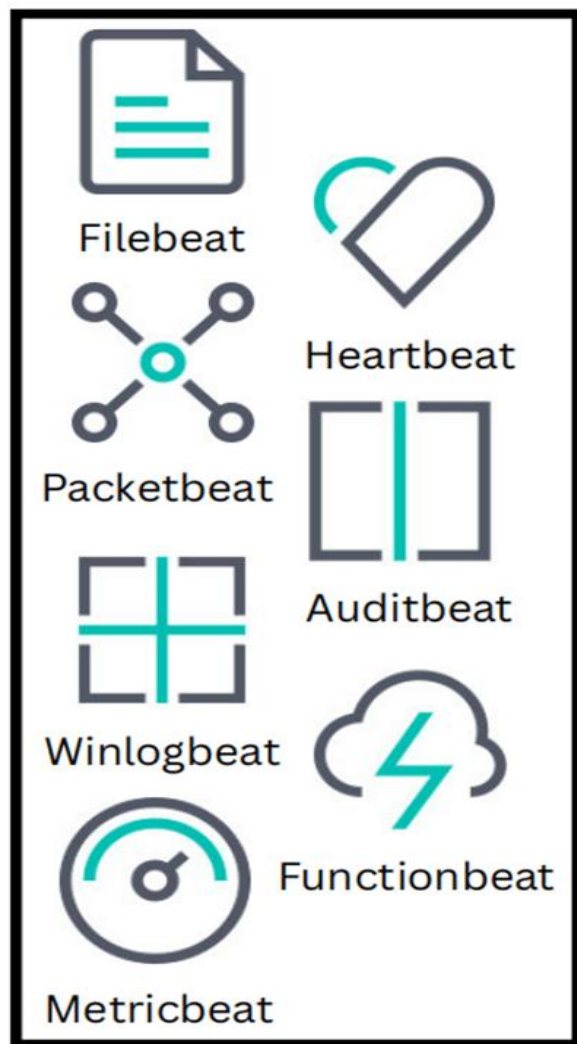
- 将分散在多个容器/节点上的日志集中收集、索引、可视化，支持快速查询和故障排查。

特性	ELK 方案	Loki+Promtail方案
组件	Elasticsearch + Logstash + Kibana	Loki + Promtail + Grafana
存储方式	全文索引（倒排索引）	仅索引标签，日志内容压缩存储
查询语言	Lucene 查询语法	LogQL（类似 PromQL）
资源占用	较高（ES 需要大量内存）	较低（无全文索引）
适用场景	复杂日志文本搜索	Prometheus 同一生态，关注标签而非全文
典型架构	Filebeat → Logstash → ES → Kibana	Promtail → Loki → Grafana

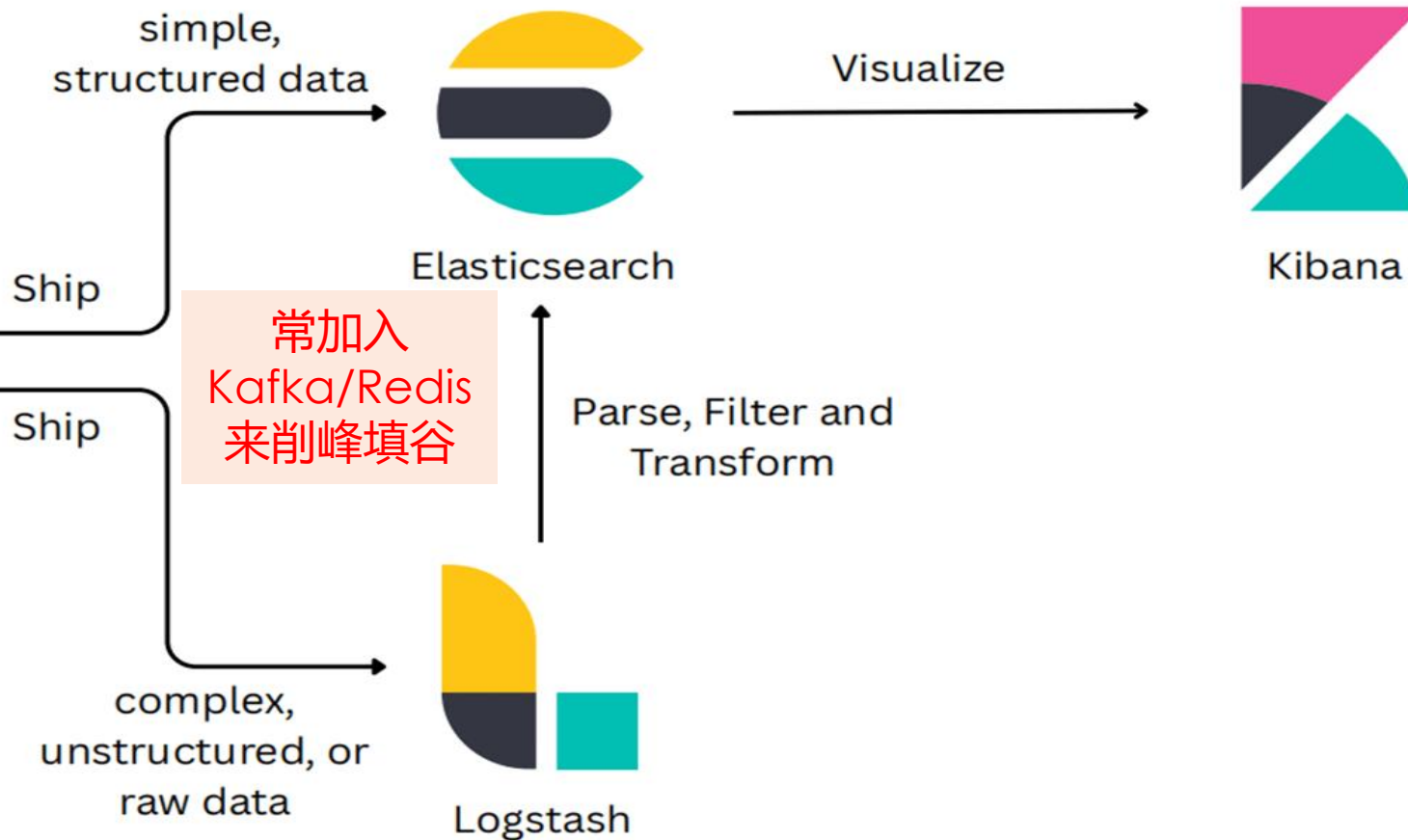


ELK 方案

Data Source



Beats





ELK配置示例

Filebeat配置示例:

Filebeat 配置文件示例: 采集日志并发送到 Kafka

filebeat.inputs:

- type: log # 输入类型: 从日志文件读取
 - enabled: true # 启用该输入
 - paths: # 日志文件路径 (支持通配符)
 - /var/log/spring-boot/*.log # 采集所有 .log 后缀的文件
 - fields: # 自定义字段, 会添加到每一条日志事件中
 - service: seckill-service # 标识日志来源的服务名称

output.kafka:

- hosts: ["kafka1:9092", "kafka2:9092"]
- topic: "app-logs" # 目标 Topic 名称
- partition.round_robin: # 分区选择策略: 轮询 (round_robin)
 - reachable_only: true # 仅将消息发送到可用的分区 (避免 Leader 不可用导致阻塞)
- compression: gzip # 启用 gzip 压缩
- max_message_bytes: 1000000 # 单条消息最大字节数

日志格式:

```
{
  "@timestamp": "...",
  "message": "实际日志内容",
  "fields": {
    "service": "seckill-service"
  }
}
```





ELK配置示例

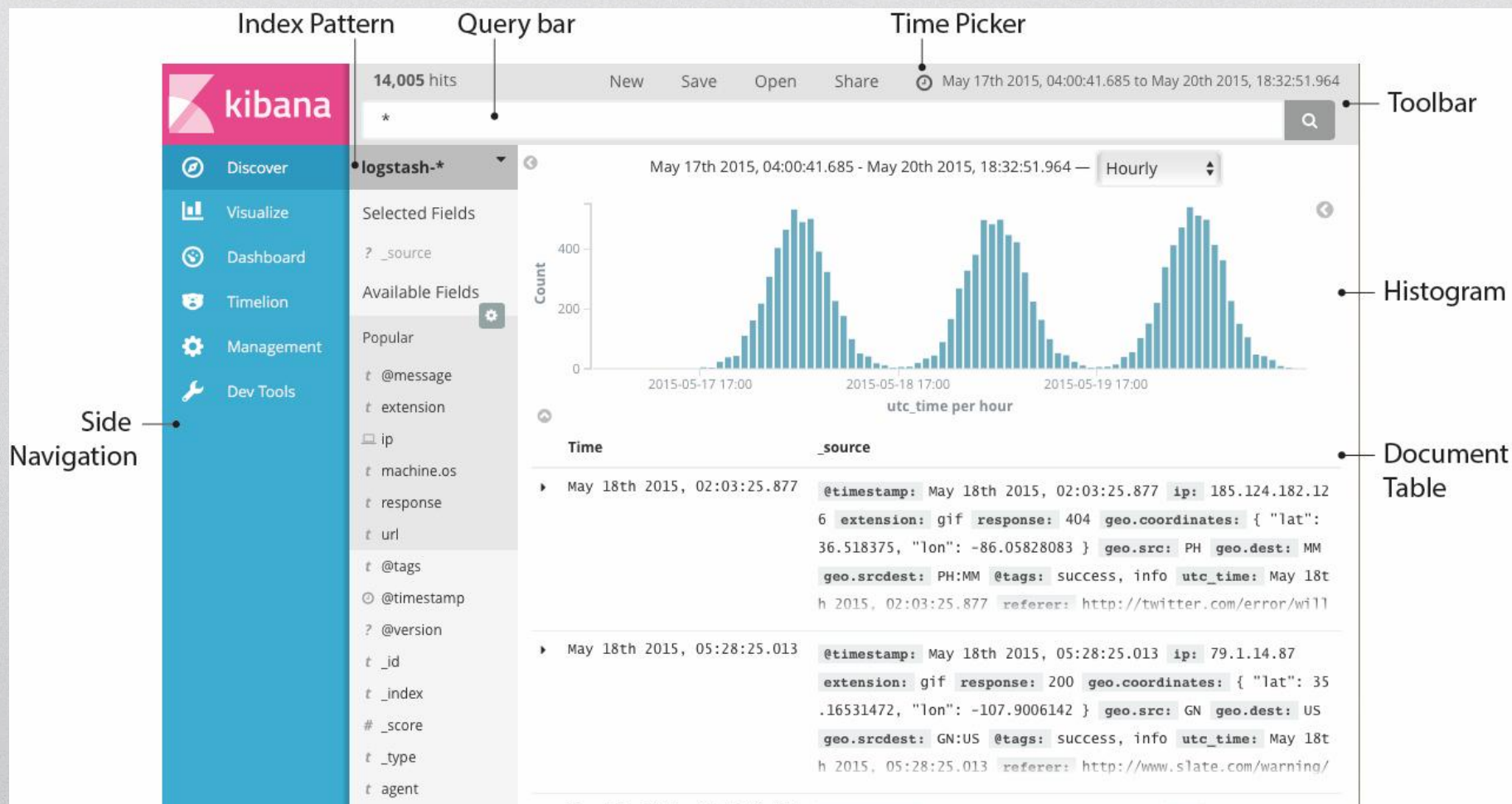
Logstash 配置示例 (logstash.conf)

```
# Logstash 管道配置: 从 Kafka 消费日志, 解析 JSON, 输出到 Elasticsearch
# =====输入配置 =====
input {
  kafka {
    bootstrap_servers => "kafka1:9092,kafka2:9092"
    topics => ["app-logs"] # 订阅的 Topic 列表 (可多个)
    group_id => "logstash-group" # 消费者组 ID
    auto_offset_reset => "earliest" # 首次消费时从最早的消息开始 (保留全部历史)
    consumer_threads => 3 # 并发消费线程数 (每个线程占用一个分区, 建议 <= 分区数)
  }
}
# ===== 过滤配置 =====
filter {
  json {
    source => "message" # 指定要解析的源字段
    # target => "parsed" # 可选: 将解析后的内容放入指定字段, 默认直接合并到根层级
    # remove_field => ["message"] # 可选: 解析后可删除原始 message 字段
  }
}
# ===== 输出配置 =====
output {
  elasticsearch {
    hosts => ["elasticsearch:9200"] # Elasticsearch 集群地址
    index => "logs-%{+YYYY.MM.dd}" # 动态索引名称: 按天切分, 例如 logs-2026.04.18
  }
}
```



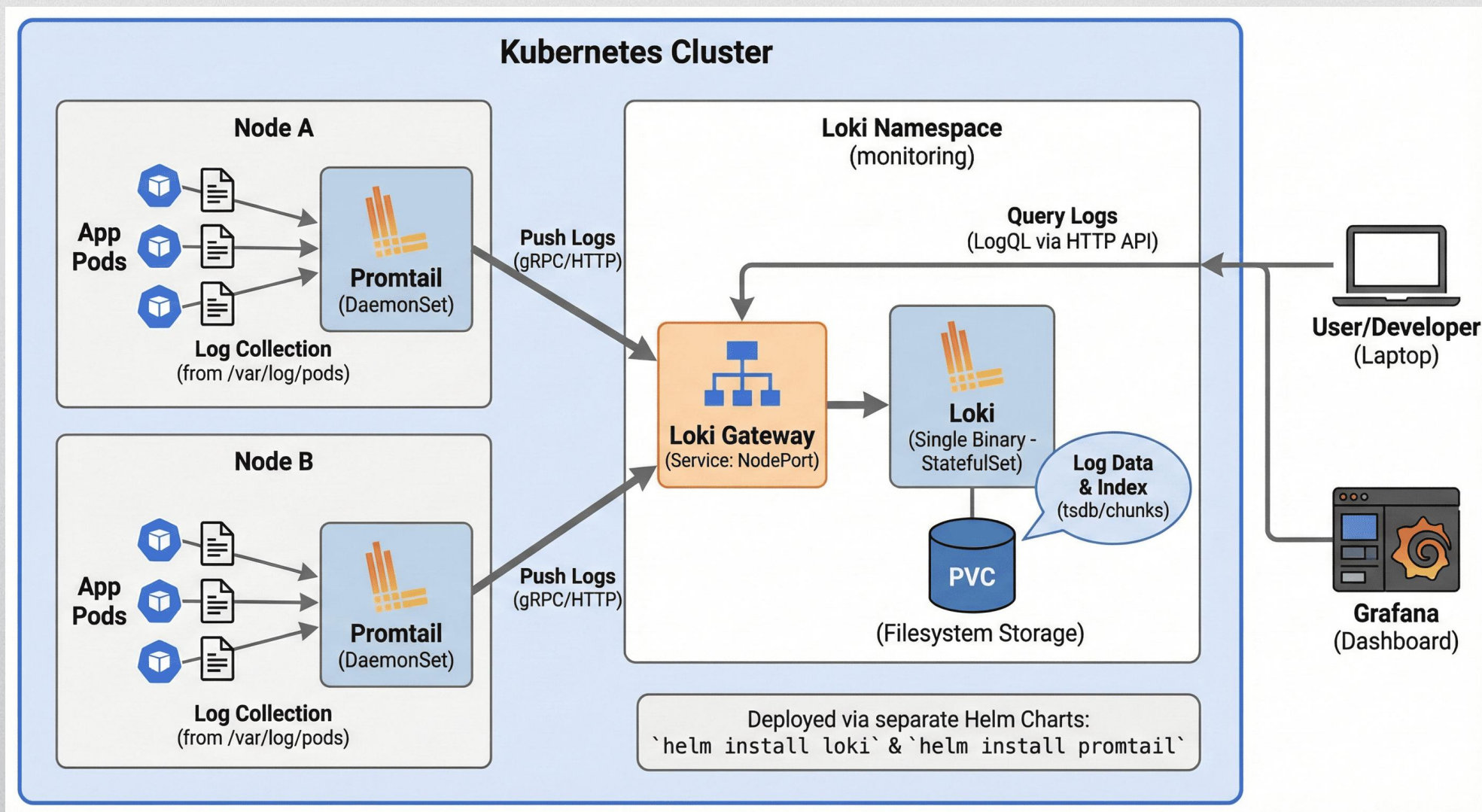

ELK配置示例

➤ 在Kibana中可视化查询查看日志





Loki+Promtail+Grafana





Promtail配置示例

Promtail 配置文件示例：采集本地日志并推送到 Loki

===== 服务端配置 =====

server:

Promtail 自身的 HTTP API 监听端口（用于健康检查、指标暴露等）

http_listen_port: 9080

gRPC 端口，设置为 0 表示禁用 gRPC（单机模式下通常不需要）

grpc_listen_port: 0

===== 位置记录 =====

positions:

记录已读取日志文件偏移量的文件路径

Promtail 重启时会读取此文件，从上次中断的位置继续采集，避免重复或遗漏

filename: /tmp/positions.yaml

===== 客户端配置 =====

clients:

Loki 的推送 API 地址（支持 HTTP/HTTPS）

- url: http://loki:3100/loki/api/v1/push

===== 抓取配置 =====

===== 抓取配置 =====

scrape_configs:

第一个抓取任务：采集系统日志

- job_name: system

静态配置（直接指定目标主机和日志路径）

static_configs:

- targets:

- localhost

目标标识（通常用于指标标签，可任意）

labels:

job: varlogs

自定义标签：任务名称

__path__: /var/log/*.log

特殊标签：指定日志文件路径（支持通配符）

第二个抓取任务：采集应用日志

- job_name: application

static_configs:

- targets:

- localhost

labels:

job: myapp

任务标签

env: production

环境标签（可用于 Loki 查询过滤）

__path__: /var/log/myapp/*.log

应用日志路径





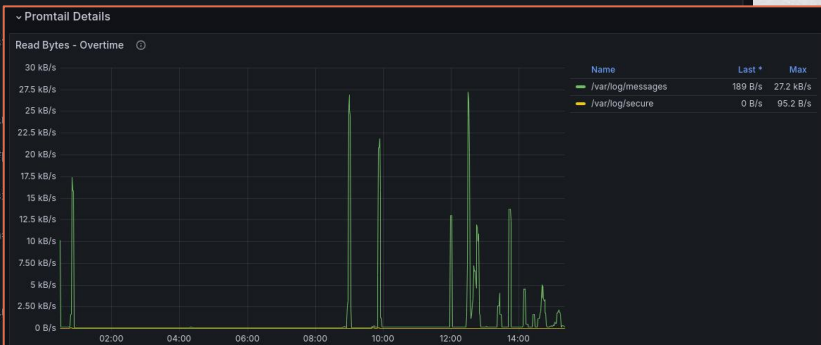
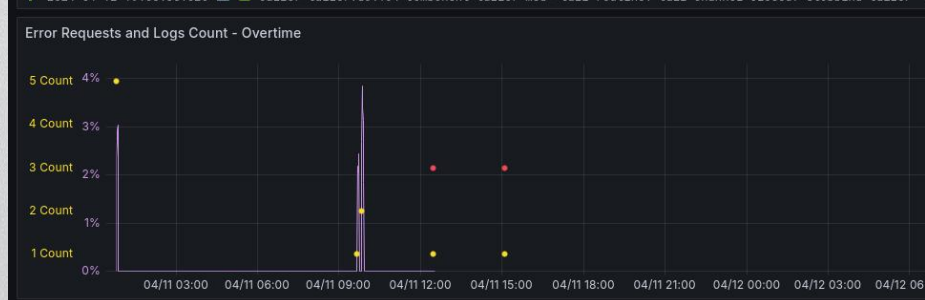
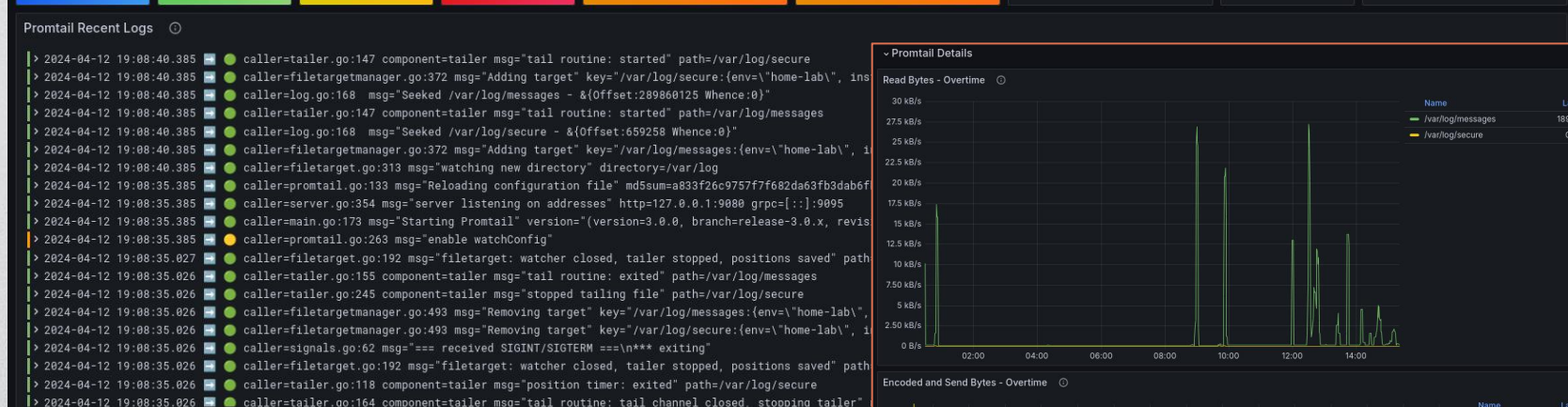
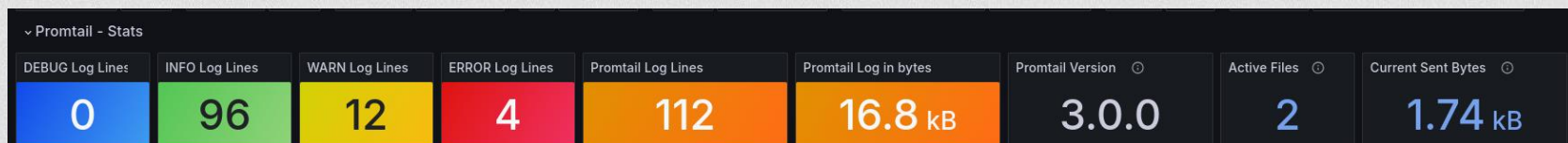
Promtail配置示例

- Promtail进行日志过滤转换等处理操作

```
scrape_configs:
- job_name: my_application
  static_configs:
    - targets: [localhost]
      labels:
        job: myapp
        __path__: /var/log/myapp/*.log
  pipeline_stages:
    # 1. 过滤: 丢弃所有 DEBUG 级别的日志
    - match:
        selector: '{job="myapp"}'
        stages:
          - regex:
              expression: '.*level=DEBUG.*'
          - drop: {}
    # 2. 解析: 通过正则提取日志级别和响应时间
    - regex:
        expression: 'level=(?P<level>\w+).*latency=(?P<latency>\d+)ms'
    # 3. 转换: 将日志级别转为小写
    - template:
        source: level
        template: '{{ ToLower .Value }}'
    # 4. 添加标签: 将 level 和 latency 设为标签
    - labels:
        level:
        latency:
```

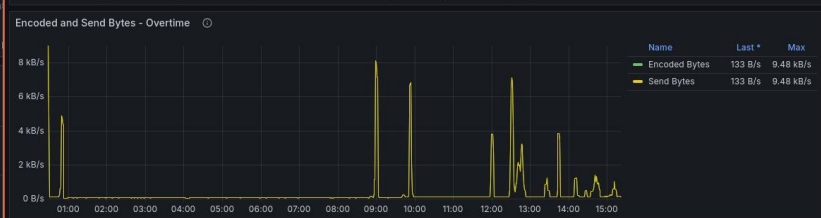



Grafana中查看日志



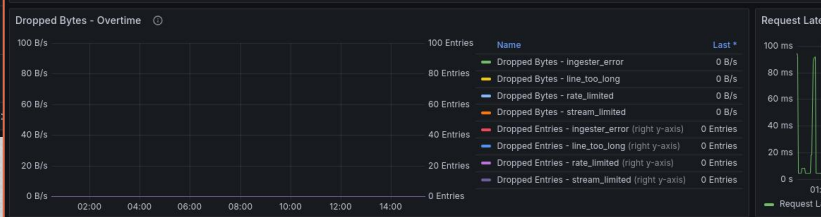
Read Bytes - Details

Path	Total Read	Current Read	Current Lines Read	Total Read in 24h
/var/log/messages	168 MB	189 B/s	0.479 Lines	41.9 MB
/var/log/secure	649 kB	0 B/s	0.0281 Lines	111 kB



Encoded and Send - Details

Loki Endpoint	Current Send Bytes	Send Bytes 24h	Current Encoded Bytes	Encoded Bytes 24h
localhost:3100	133 B/s	14.6 MB	133 B/s	14.6 MB



<https://voidquark.com/blog/promtail-grafana-dashboard/>



武汉大学
WUHAN UNIVERSITY

3、指标监控



指标 (Metrics)

➤ 定义

- 指标 (Metrics) 是随时间变化的数值统计型数据，用于描述系统的状态和行为。

➤ 关键指标模式 (Google 四大黄金指标)

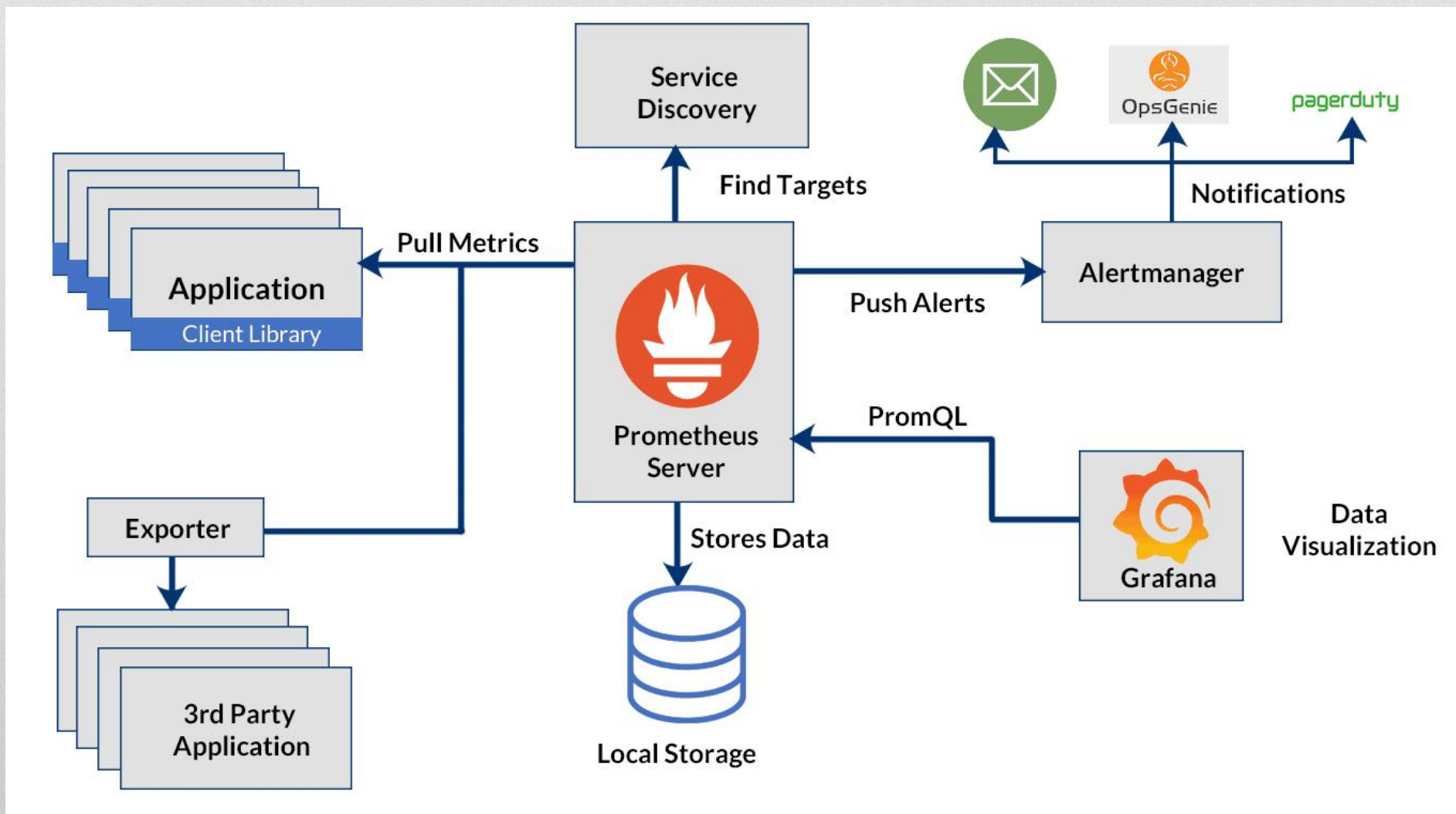
- 延迟：服务处理请求所需时间，常用百分位数值（如P95、P99、P99.9指标）
- 流量：系统承载的请求量（QPS、TPS）。
- 错误：请求失败率（5xx、超时等）。
- 资源饱和度：资源使用程度（CPU、内存、连接数）。

➤ 典型实现

- Prometheus（拉取指标数据）+ Grafana（可视化）。



Prometheus+Grafana方案





Prometheus: 云原生监控的事实标准

➤ 核心组件

- Exporters: 暴露指标 (Node Exporter、Blackbox Exporter、JMX Exporter)
- Prometheus Server: 拉取指标、存储时序数据、执行 PromQL 查询。
- Alertmanager: 管理告警 (去重、分组、路由、静默) 。
- Grafana: 可视化仪表盘。

➤ 拉模型 (Pull) 优势

- 主动探测目标健康状态。
- 可配置服务发现 (K8s、Consul) 。
- 更容易做水平扩展 (通过分片) 。

➤ 时序数据库

- 内置TSDB数据库, 可支持InfluxDB、Cortex 等专业时序数据库
- 按时间分块存储 (每 2 小时一个 block) , 高压缩率 (每个样本约 1-2 字节) 。
- 支持降采样 (需 Thanos/Cortex 存储) 。





Spring Boot 集成 Micrometer + Prometheus

使用Micrometer的@Timed注解，监控服务的P50、P95、P99延迟指标

```
@Configuration
public class MetricsConfig {
    @Bean
    public TimedAspect timedAspect(MeterRegistry registry) {
        return new TimedAspect(registry);
    }
}
```

```
@RestController
public class OrderController {

    @PostMapping("/order")
    @Timed(value = "order.create.duration",
          histogram = true,
          percentiles = {0.5, 0.95, 0.99},
          extraTags = {"region", "cn-east"})
    public Order createOrder(@RequestBody OrderRequest request) {
        // 模拟业务耗时 (随机 10~200ms)
        sleepRandom();
        return new Order(...);
    }
}
```

通过URL /actuator/prometheus可以看到指标:

```
# HELP order_create_duration_seconds
# TYPE order_create_duration_seconds histogram
order_create_duration_seconds_bucket{le="0.005",} 0.0
order_create_duration_seconds_bucket{le="0.01",} 2.0
order_create_duration_seconds_bucket{le="0.025",} 5.0
order_create_duration_seconds_bucket{le="0.05",} 20.0
order_create_duration_seconds_bucket{le="0.1",} 80.0
order_create_duration_seconds_bucket{le="0.25",} 95.0
order_create_duration_seconds_bucket{le="0.5",} 99.0
order_create_duration_seconds_bucket{le="1",} 100.0
order_create_duration_seconds_bucket{le="+Inf",} 100.0
order_create_duration_seconds_count 100.0
order_create_duration_seconds_sum 12.345
```





Spring Boot 集成 Micrometer + Prometheus

全局配置

global:

```
scrape_interval: 15s      # 全局抓取间隔, 默认 1m
evaluation_interval: 15s  # 规则评估间隔, 默认 1m
scrape_timeout: 10s      # 抓取超时时间, 默认 10s
```

告警规则文件路径 (支持通配符)

rule_files:

```
- "rules/*.yml"
```

抓取配置

scrape_configs:

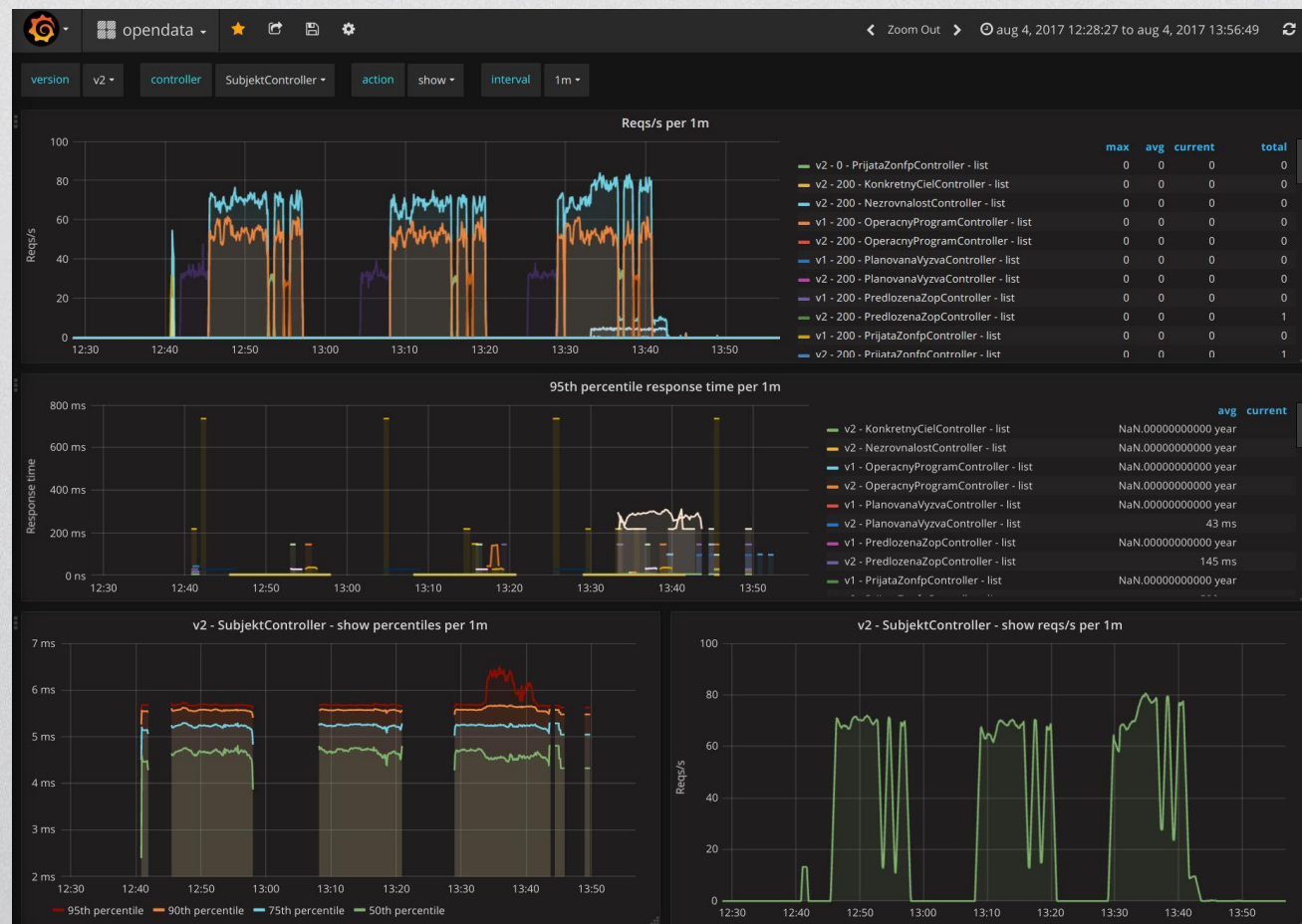
```
# 第一个抓取任务: 抓取 Prometheus 自身指标
```

```
- job_name: 'prometheus'
  static_configs:
    - targets: ['localhost:9090']
```

```
# 第二个抓取任务: 抓取自定义应用指标
```

```
- job_name: 'order-service'
  metrics_path: '/actuator/prometheus'
  static_configs:
    - targets: ['order-service:8080', 'order-service:8081']
    labels:
      env: 'prod'
      service: 'order'
```

Prometheus的配置



Grafana可视化展示指标



Spring Boot 集成 Micrometer + Prometheus

Prometheus中设置报警规则

```
groups:
- name: latency_alerts
  rules:
    # 告警: 订单创建 P99 延迟超过 500ms 持续 5 分钟
    - alert: HighOrderCreationLatency
      expr: |
        histogram_quantile(0.99,
          sum(rate(order_create_duration_seconds_bucket[5m])) by (le, application)
        ) > 0.5
      for: 5m
      labels:
        severity: warning
        service: order
      annotations:
        summary: "订单服务 P99 延迟过高"
        description: "服务 {{ $labels.application }} 的 P99 延迟为 {{ $value | humanize }}s, 超过 0.5s 已持续 5 分钟。"

    # 告警: 延迟比过去 1 小时增加超过 100% (动态基线示例)
    - alert: LatencySpike
      expr: |
        (
          histogram_quantile(0.99, sum(rate(order_create_duration_seconds_bucket[5m])) by (le))
          /
          histogram_quantile(0.99, sum(rate(order_create_duration_seconds_bucket[5m] offset 1h)) by (le))
        ) > 2
      for: 2m
      labels:
        severity: warning
      annotations:
        summary: "订单服务延迟突然翻倍"
        description: "当前 P99 延迟是 1 小时前的 {{ $value | humanize }} 倍"
```


4、链路追踪



链路追踪 (Tracing)

➤ 定义

- 链路追踪记录请求在分布式系统中的完整调用路径，包括每个服务、每个操作的耗时和状态。
- 当你发现一个请求慢了，你无法知道它到底慢在哪个环节。是网关慢？订单服务慢？还是调用的库存服务慢？甚至是数据库慢？链路追踪就是来解决这个问题的。

➤ 为什么需要链路追踪？

- 微服务数量多，调用链复杂 ($A \rightarrow B \rightarrow C \rightarrow D \dots$)
- 传统日志无法关联同一个请求在不同服务上的日志
- 指标监控只能看到整体趋势，无法定位具体哪个环节慢。

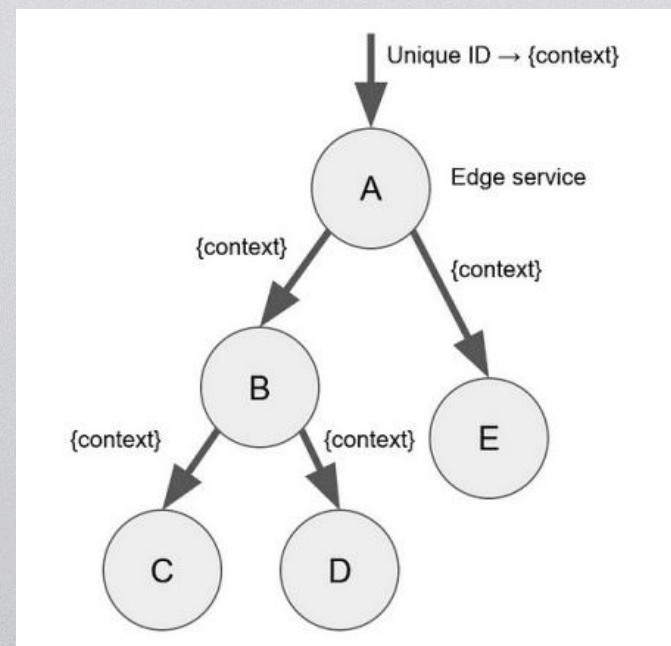
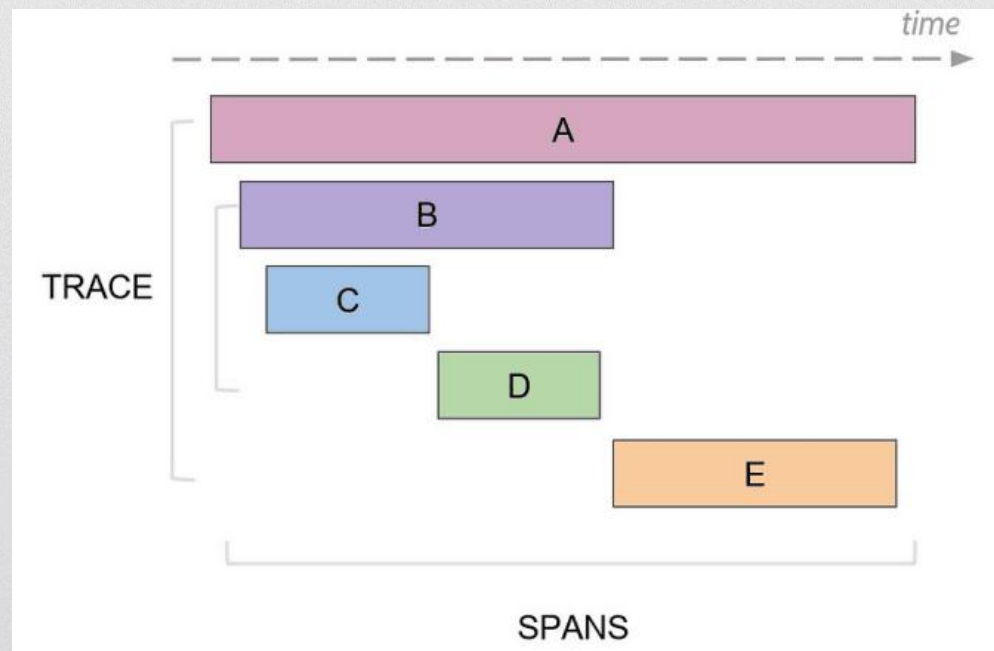


链路追踪 (Tracing)



核心概念

- **Trace**: 每个请求分配一个全局 TraceId 标识, 这个 ID 会贯穿所有服务。
- **Span**: 每个服务的处理过程 (如一个服务调用、一次数据库查询) 记录为一个 Span, 包含开始时间、结束时间、元数据。
- **Parent-Child 关系**: 调用方 Span 和被调用方 Span 的可以嵌套, 形成调用树结构。
- **上下文传播**: 通过 HTTP Header 将 TraceId 跨服务传递。





链路追踪标准

➤ 早期碎片化问题

- 各家有不同实现 (Zipkin、Jaeger、SkyWalking) , 无法互通。

➤ OpenTelemetry标准

- CNCF 下的统一可观测性标准, 是当前的事实标准, 定义了数据模型和传播协议。
 - Trace 数据模型: 定义 Trace、Span、SpanContext 的结构。
 - 传播器 (Propagator) : 定义如何通过 HTTP Headers 传递 Trace 上下文。

➤ W3C TraceContext 标准

- HTTP 层面的标准

请求从服务 A 到服务 B 时

- A 在 HTTP 请求头中添加traceparent。格式: 版本-TraceId-SpanId-TraceFlags, 如00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01
- B 接收到后, 就知道自己属于哪个 Trace, 并把自己的 SpanId 作为 Child Span 记录。





链路追踪系统选型

特性	Jaeger	SkyWalking	Zipkin
定位	纯链路追踪系统	应用性能管理(APM)平台	轻量级追踪系统
特点	云原生	功能全面、无侵入探针、APM能力	轻量、简单、易部署
语言与侵入性	支持多语言，部分侵入	无侵入（通过Java Agent）	支持多语言，侵入性中等
存储后端	ES, Cassandra, Kafka等	ES, MySQL, TiDB, BanyanDB	ES, MySQL, Cassandra
告警功能	需额外组件	内置	无
主要优势	•性能优秀，支持高并发 •云原生集成度高 •自适应采样	•功能最全 •零代码侵入 •中文社区活跃	•部署和集成简单 •资源占用小 •与Spring Cloud集成良好
主要劣势	•配置相对复杂 •纯追踪，缺少APM功能	•资源消耗较高 •部署较复杂	•UI功能基础 •查询性能在大数据量下受限 •缺少高级分析功能



Spring Boot 集成 Sleuth + Zipkin

#application.yml

spring:

 application:

 name: order-service

 zipkin:

 base-url: <http://zipkin:9411> # Zipkin 服务地址 (或 Jaeger 的 collector 地址)

 sender:

 type: web

 # 使用 HTTP 上报, 也可用 kafka、rabbit

 sleuth:

 sampler:

 probability: 1.0

 # 采样率, 生产环境建议 0.1~0.5

 traceId128Bit: true

 # 使用 128 位 TraceId

Spring Boot自动生成并上报链路数据

#logback-spring.xml

<pattern>

 %d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36}

 [%X{traceId:-},%X{spanId:-}] - %msg%n

</pattern>





Spring Boot 集成 Sleuth + Zipkin

```
import brave.Tracer;
```

```
@Service
```

```
public class OrderService {
```

```
    @Autowired
```

```
    private Tracer tracer;
```

```
    public Order getOrderWithDetails(Long orderId) {
```

```
        Span newSpan = tracer.nextSpan().name("query-mysql").start();
```

```
        try (Tracer.SpanInScope ws = tracer.withSpanInScope(newSpan)) {
```

```
            // 执行数据库查询
```

```
            Order order = orderRepository.findById(orderId);
```

```
            return order;
```

```
        } finally {
```

```
            newSpan.finish();
```

```
        }
```

```
    }
```

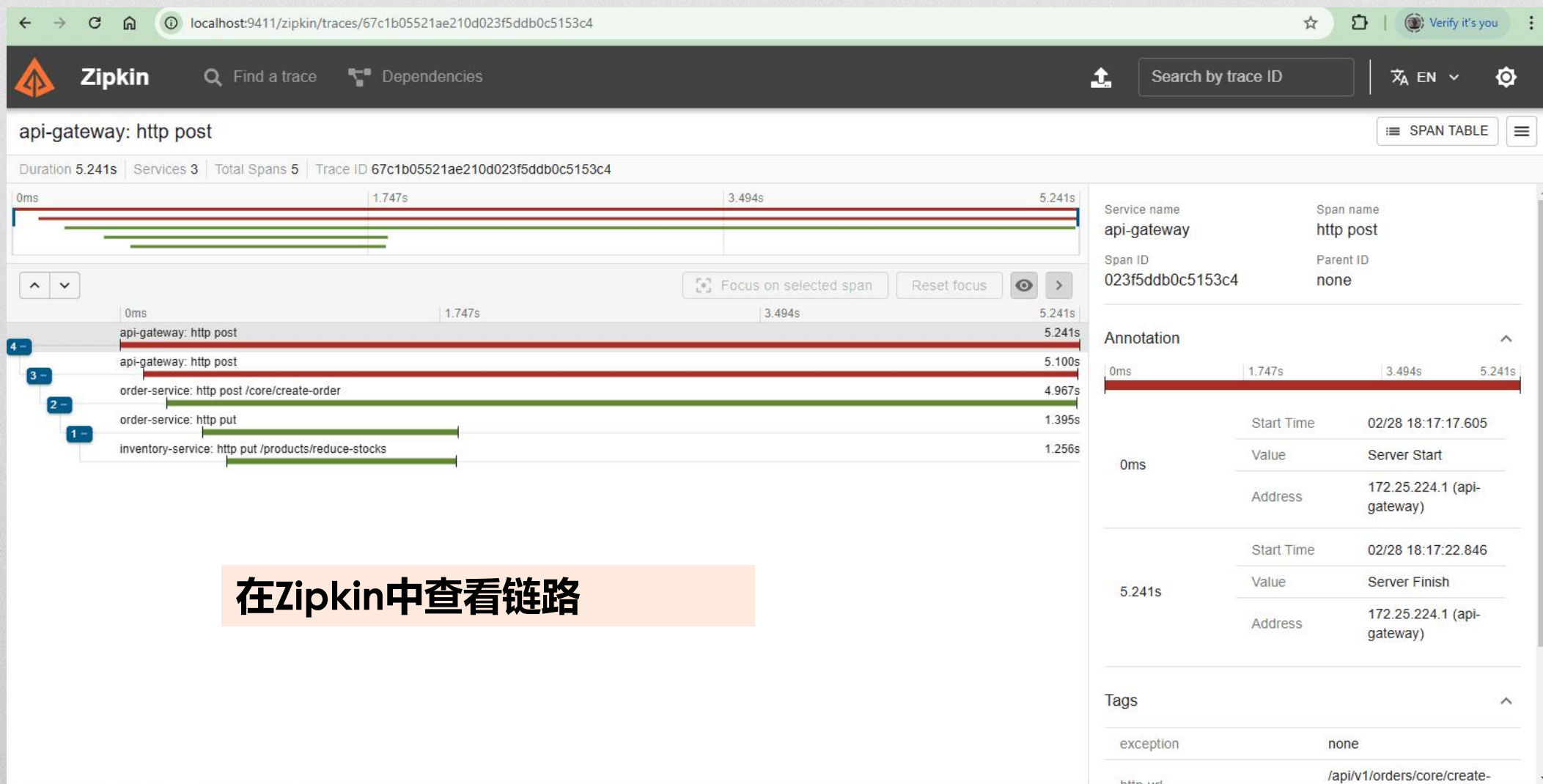
```
}
```

Spring Boot中可手动创建span，跟踪内部操作
(如数据库查询、缓存访问)





Spring Boot 集成 Sleuth + Zipkin



在Zipkin中查看链路



三位一体：通过 Traceld 串联日志、指标和链路

- 场景：指标显示 P99 延迟升高，我们想找到导致慢的具体请求。

关联	实现方式
指标 → 链路	在指标中记录慢请求的 Traceld 分布（如使用 Histogram 记录 Traceld），或通过持续分析（Continuous Profiling）
日志 → 链路	日志中打印 Traceld，通过 Traceld 查询日志系统
链路 → 日志	在 Zipkin/Jaeger UI 中点击 Span，直接跳转到对应服务的日志（需集成 Loki 或 Elasticsearch）

5、服务网格



服务网格：将微服务治理下沉到基础设施层

➤ 传统微服务治理的痛点

- 每个服务都要集成 SDK（如 Spring Cloud、Resilience4j）实现熔断、重试、服务发现等功能。
- 多语言场景下，每种语言都要实现相同的治理逻辑，维护成本高。
- SDK 升级需要重新构建和部署所有服务，耦合性强。

➤ 服务网格的核心理念

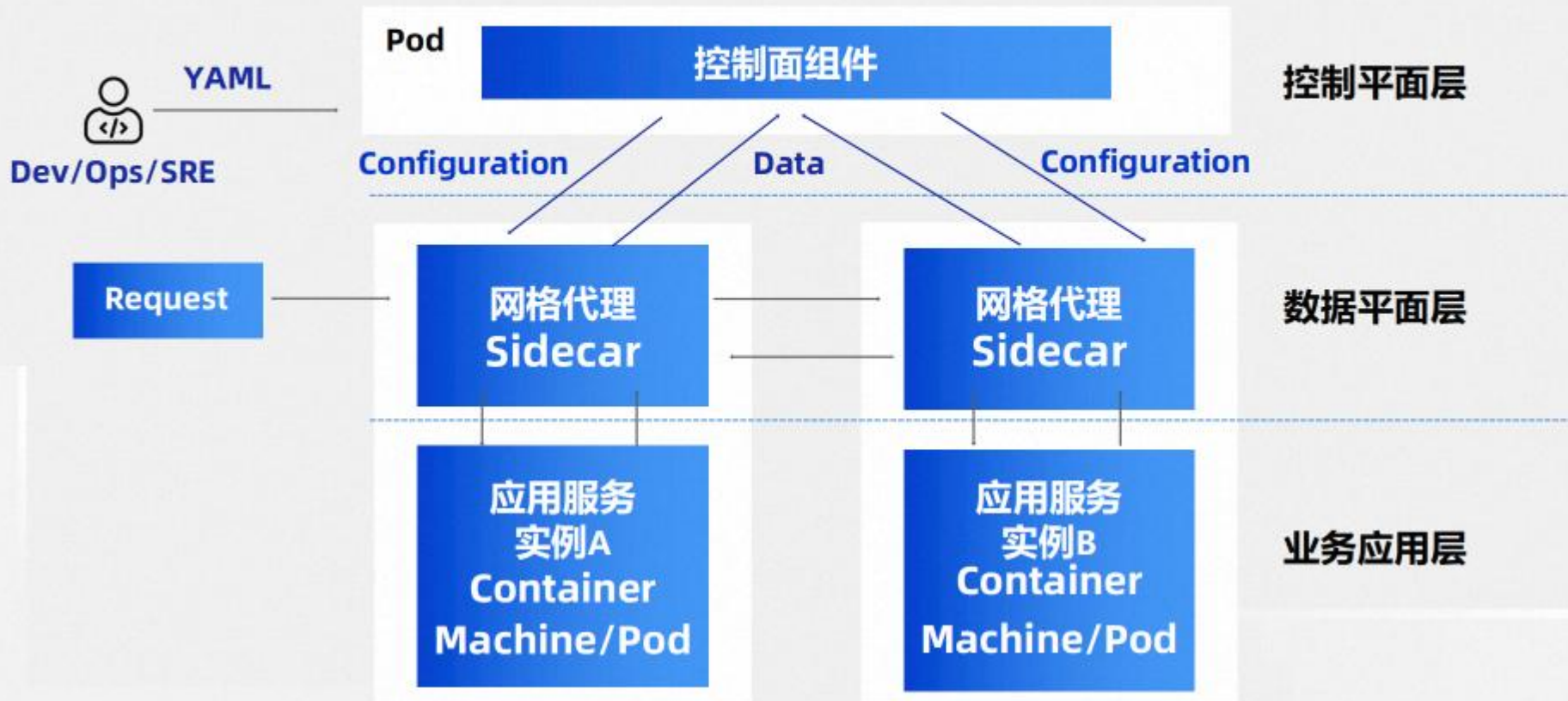
- 将流量治理（熔断、重试、路由、可观测性）从应用代码中抽离，下沉到基础设施层。
- 通过 Sidecar 代理（如 Envoy）拦截所有进出流量，应用代码无需感知治理逻辑。



典型架构

服务网格经典Sidecar模式

控制面与数据面的组件都运行于一个K8s集群中





服务网格选型

特性	Istio	Linkerd
数据平面	Envoy (C++)	Linkerd2-proxy (Rust)
控制平面	Istiod (Go)	轻量控制器
资源消耗	较高	较低
多语言支持	☒ 任何语言	☒ 任何语言
流量治理能力	丰富 (路由、熔断、重试、镜像)	基础 (负载均衡、重试)
mTLS 自动加密	☒	☒
可观测性	丰富 (Prometheus、Jaeger、Kiali)	基础
学习曲线	陡峭	平缓
生产案例	众多大型企业	较新但增长快



Istio：最流行的服务网格实现



数据平面

- Sidecar 模式：为集群中启动的每个 Pod 都部署一个 Envoy 代理，或者与在虚拟机上运行的服务并行运行一个 Envoy 代理。
- Ambient 模式：每个节点上使用四层代理，另外可以选择为每个命名空间使用一个 Envoy 代理来实现七层功能。



控制平面 (Istiod)

- Pilot：提供服务发现、流量管理配置下发（路由、熔断、重试等）。
- Citadel：提供服务间 mTLS 认证和授权。
- Galley：配置验证和分发（已合并到 Istiod）。
- Telemetry：自动生成可观测性数据（Prometheus 指标、Jaeger 追踪）。

<https://istio.io/latest/zh/docs/overview/what-is-istio/>



6、无服务器架构



无服务器架构 (Serverless)

➤ 定义

- 无服务器架构是一种云计算执行模型，开发者编写代码并上传到平台，平台自动管理服务器、伸缩、高可用和计费，开发者无需关心任何基础设施。
- 不是“没有服务器”，而是“**开发者无需感知服务器**”。

➤ **Serverless = FaaS + BaaS**

- FaaS（函数即服务）：运行无状态的、事件驱动ed的函数代码，如 AWS Lambda。
- BaaS（后端即服务）：使用云托管的第三方服务，如云数据库、开放认证、云存储。



无服务器架构

维度	传统架构（自建服务器）	容器化架构（K8s）	无服务器架构（Serverless）
服务器管理	自己买/租服务器	集群管理，仍需关注节点	完全无感知
弹性伸缩	手动或需预置	自动（基于指标）	自动（从 0 到 N）
计费粒度	按服务器/实例小时	按 Pod 资源小时	按函数调用次数 + 执行时间
闲置成本	有（预留资源）	有（最小副本）	无（缩到 0 时不计费）
冷启动	无	有（Pod 启动）	有（函数实例创建）
运维复杂度	高	中	极低



Serverless的应用场景

场景	说明	示例
Web/API 后端	构建 RESTful API 或 GraphQL 端点, 自动伸缩	电商网站的订单查询接口
事件处理	响应对象存储、数据库变更、消息队列等事件	图片上传后自动生成缩略图
定时任务	按 Cron 表达式执行定期任务	每日凌晨清理过期数据
数据处理流水线	组合多个函数处理流式数据	实时日志分析、ETL
移动应用后端	为移动 App 提供认证、数据同步、推送	小程序后端 (腾讯云 SCF)
机器人/聊天机器人	处理 webhook 请求	Slack、钉钉机器人
机器学习推理	轻量级模型推理 (需 GPU 函数)	图片分类、OCR
全栈 Web 应用	前端静态托管 + BaaS + FaaS	Jamstack 应用 (如 Gatsby + AWS Lambda)



函数即服务 (FaaS)

➤ 定义

- FaaS 是一种云服务，允许开发者上传函数代码，平台在事件触发时执行函数，自动管理运行时环境、伸缩、高可用。
- Scale-to-zero: FaaS 平台的一项核心能力，指当函数或服务没有请求时，平台自动将运行实例数量缩减为 0，不占用任何计算资源。

➤ 真实案例

- Netflix: 使用 AWS Lambda 进行媒体编码、日志处理等后台任务。
- Coca-Cola: 使用 Azure Functions 搭建自动化营销系统。
- iFood (拉美外卖平台): 使用 AWS Lambda 处理订单事件，峰值 2 万请求/秒。
- 微信小程序: 大量开发者使用腾讯云 SCF 作为无服务器后端。



函数生命周期

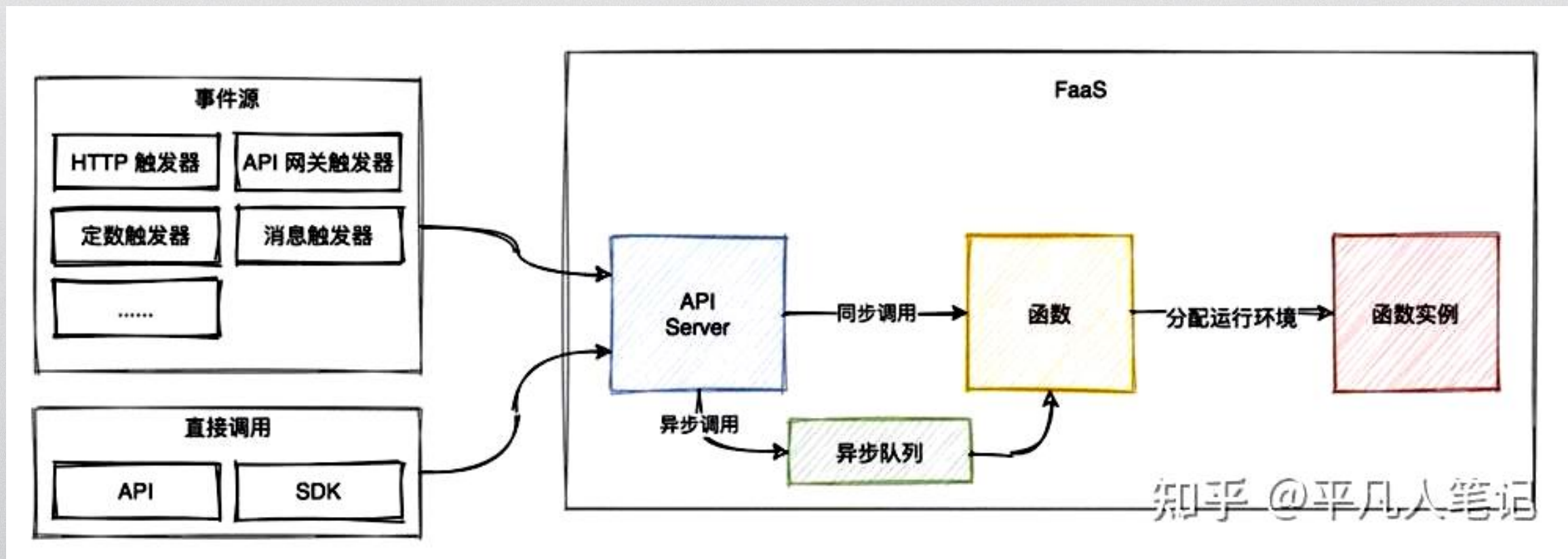


函数生命周期

- 编写函数：使用支持的编程语言（Node.js、Python、Java、Go 等），实现处理逻辑。
- 部署：上传代码或容器镜像到云平台。
- 配置触发器：绑定事件源（HTTP 请求、消息队列、对象存储事件、定时器等）。
- 执行：事件发生时，平台自动拉取函数实例并执行。
- 伸缩：并发请求增加时，平台自动创建更多实例（每个实例处理一个请求或根据平台配置）。
- 空闲回收：函数闲置一段时间后，实例被销毁，下次调用需要冷启动。



FaaS的实现原理





主流 FaaS 平台对比

➤ 在线商业化FaaS平台

- AWS Lambda: 生态最成熟, 文档丰富, 适合全球业务或已在 AWS 上。
- Azure Functions: 与 Visual Studio 集成好, 适合 .NET 开发者。
- Google Cloud Functions: 与 Firebase 无缝集成, 适合移动/Web 应用。
- 阿里云函数计算: 国内用户首选, 与阿里云产品集成好, 有免费额度。

➤ 开源FaaS平台

- Knative: Kubernetes 原生, 需要 Istio。功能最全, 企业级, 但重。
- OpenFaaS: 最轻量, 开发友好, 快速上手。
- OpenWhisk: 支持复杂函数编排, 适合工作流场景。



阿里云函数计算

☰

阿里云

工作台

账号全部资源

华东1 (杭州)

搜索...

执行成功!

▼ 详细信息

返回结果 日志输出 请求信息 执行摘要

Hello, World!

请求 ID 1-69e48f47-15581e26-4f684d2e746d
实例 ID c-69e48eff-151a4efa-689ea1b7b9b9

▲ 隐藏详细信息

测试函数

实时日志

部署代码

File Edit Selection View Go Run Terminal Help

app.py — code — OpenVSCode Server

EXPLORER

CODE

app.py

HELP.md

app.py

Flask

```
1 from flask import Flask
2 from flask import request
3
4 app = Flask(__name__)
5
6 @app.route('/', defaults={'path': ''})
7 @app.route('/<path:path>', methods=['GET', 'POST', 'PUT', 'DELETE'])
8 def hello_world(path):
9     data = request.stream.read()
10     print("Path: " + path)
11     print("Data: " + str(data))
12     return "Hello, World!"
13
14 if __name__ == '__main__':
15     app.run(host='0.0.0.0', port=9000)
16
```




什么是 BaaS? —— 后端按需即用

➤ 定义

- 后端即服务 (Backend as a Service, BaaS) 是一种云服务模式, 开发者通过 SDK 或 API 直接使用预置的云托管后端组件, 无需自行搭建和维护服务器。

➤ BaaS vs FaaS:

- BaaS: 提供预构建的后端服务 (数据库、认证、存储), 开发者直接调用 API 使用。
- FaaS: 提供函数执行环境, 开发者上传自定义代码片段, 由事件触发运行。
- 两者共同构成 Serverless 架构, 解决后端开发的不同层面的问题。



BaaS 的核心服务

后端服务	说明	典型产品
云数据库	托管数据库，支持 NoSQL 或 SQL，自动备份、扩容	Firestore、Supabase (PostgreSQL)、MongoDB Atlas
用户认证	用户注册、登录、OAuth 集成 (Google、GitHub 等)、JWT 管理	Firebase Auth、Auth0、Supabase Auth
云存储	文件上传、下载、CDN 加速，支持图片处理	Firebase Storage、AWS S3
云函数	在 BaaS 平台内编写和执行自定义后端逻辑 (与 FaaS 重叠)	Cloud Functions for Firebase
实时数据库	基于 WebSocket 的数据实时同步，多端自动更新	Firebase Realtime Database、Supabase Realtime
消息推送	向移动设备推送通知	Firebase Cloud Messaging (FCM)
静态托管	托管前端静态页面，自带 CDN 加速	Firebase Hosting





BaaS 平台对比

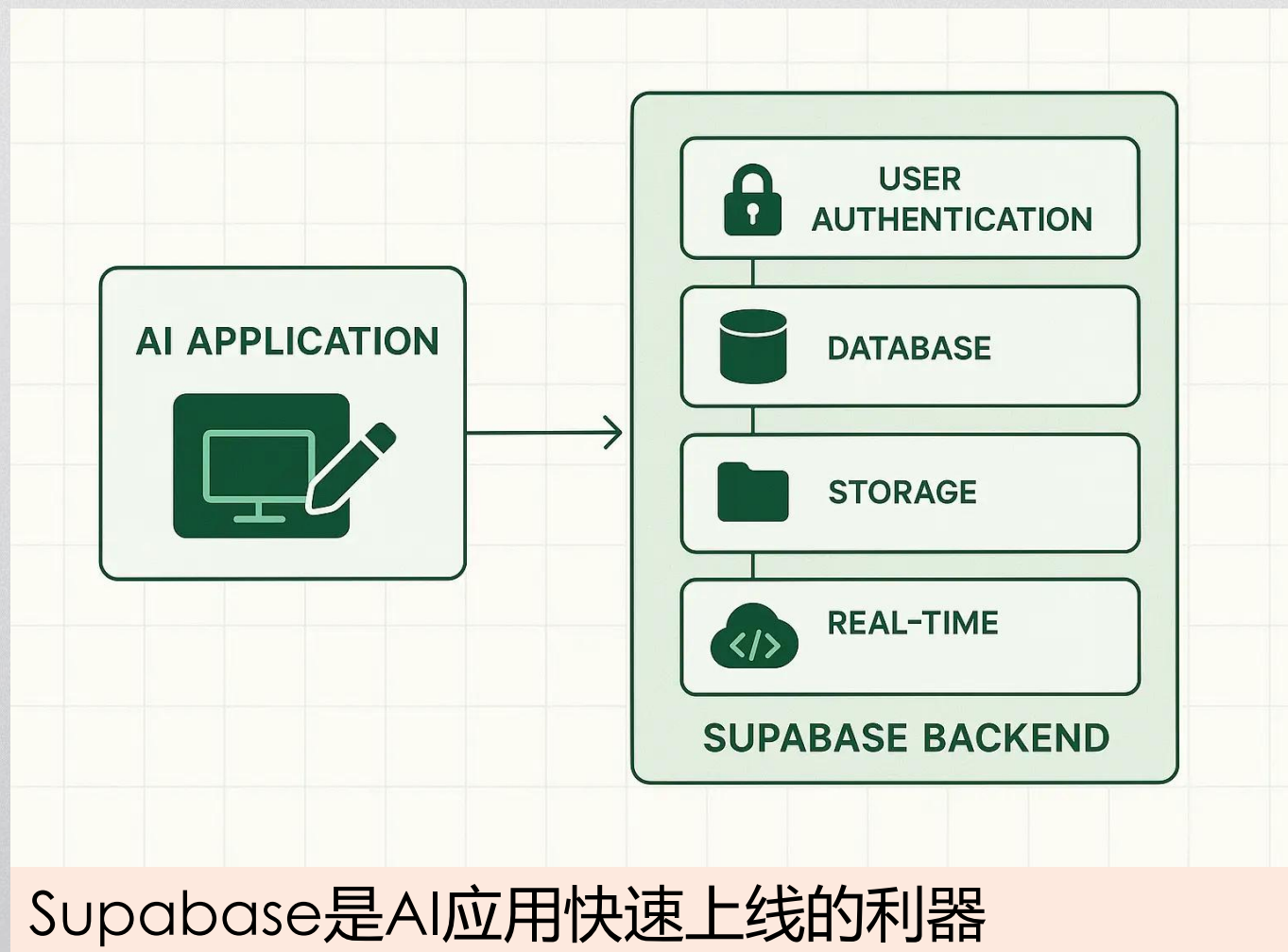
维度	Firebase (Google)	Supabase (开源)	AWS Amplify	腾讯云 CloudBase
开源/闭源	闭源, 专有	☑完全开源	闭源	闭源
自托管	✗不支持	☑支持	✗不支持	✗不支持
数据库	Firestore (NoSQL)	PostgreSQL (SQL)	DynamoDB / DataStore	云数据库 (MongoDB)
实时能力	☑原生强实时	☑监听 PostgreSQL 变更, 高并发	有限	☑支持
主要优势	生态成熟, 实时强	开源、SQL、可控	与 AWS 生态深度集成	微信生态、合规、国内低延迟



开源 BaaS 平台Supabase

➤ Supabase平台

- 核心是 PostgreSQL, PostgREST 自动将数据库表转换为 RESTful API, 零代码实现 CRUD。
- GoTrue 提供 JWT 用户认证, 与 PostgreSQL 的行级安全 (RLS) 深度集成。
- Storage内置对象存储Minio
- Realtime 监听 PostgreSQL, 用 WebSocket 推送数据变更。





开源 BaaS 平台 Supabase

➤ 示例：数据查询和事件订阅

```
// 初始化 Supabase 客户端
import { createClient } from '@supabase/supabase-js'
const supabase = createClient('https://your-project.supabase.co', 'your-public-api-key')

// 查询数据 (自动生成的 REST API)
const { data, error } = await supabase.from('users').select('*')

// 实时订阅数据变更
supabase
  .channel('users-channel')
  .on('postgres_changes', { event: 'INSERT', schema: 'public', table: 'users' }, (payload) => {
    console.log('New user:', payload.new)
  })
  .subscribe()
```